

プログラミングI 演習

Computer Programming I: Exercises

第6回 関数

担当

A-P1 辻 愛里

A-P2 北 直樹

A-P3 清水 昭伸

1

本日の講義

本日の目標

- 関数の役割を知ろう
- プロトタイプ宣言を知ろう
- ヘッダファイルの役割を知ろう
- 関数を使った簡単なプログラムを作成しよう

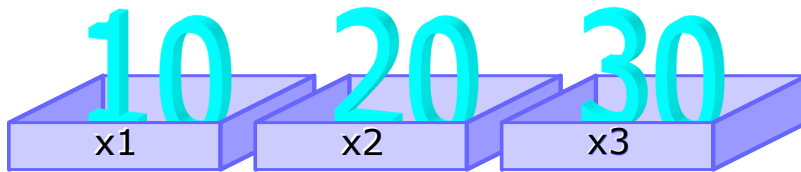
本日の資料

1. 前回のおさらい
2. 関数と変数の局所性
3. 演習課題

1. 前回のおさらい

配列: 同じ型の変数をまとめて扱う

複数の変数 (`int x1, x2, x3;`)



1つの名前に1つの領域が割り当てられる

お互いの関係
「隣の変数」や「次の変数」
等の情報をもっていない

配列: 「`int x[3];`」



1つの名前に「n個の」領域が割り当てられる
(例の場合は「n=3」)

配列「x」の「i」番目の要素
という考え方ができる

3

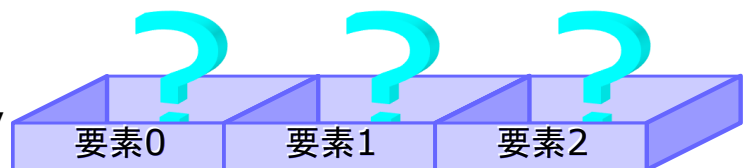
1. 前回のおさらい

配列の宣言

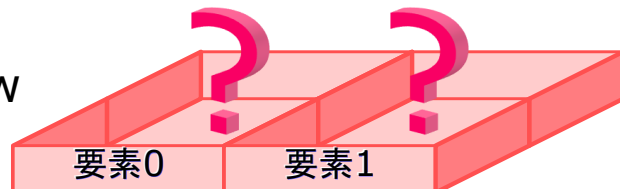
型 配列名 [配列の大きさ];

例: `int v[3];`
`double w[2];`

配列v



配列w



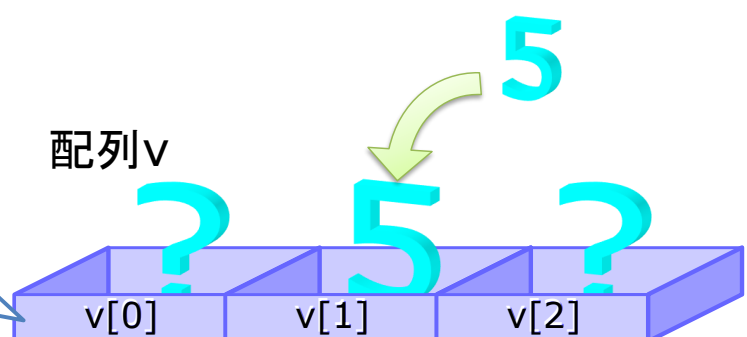
各要素へのアクセス

配列名 [要素番号] = 5;

配列「v」の
1番目の要素 → `v[1]=5;`

配列の要素は0から始まる

配列v



4

1. 前回のおさらい

演習5-3 数列に関する課題解決(添字に注意)

$$a_n = 7 \cdot 3^{n-1} - 2 \text{ です}$$

配列を使うと一部の問題が楽に解ける. 解いてみよう

[1] 数列 $\{a_n\}$ において, $a_1=5$, $a_{n+1}=3a_n+4$ のとき, 第20項(a_{20})は何?

[2] 数列 $\{a_n\}$ が $a_n=3^n-30$ で表されるとき, 初項(a_1)から第20項(a_{20})までの和は?

$$S_n = (3/2) \cdot (3^n - 1) - 30n \text{ です}$$

累乗の計算を練習しよう

[3] 3で割り切れないすべての正の整数を, 小さいものから並べてできる数列を $a_1, a_2, a_3, \dots, a_n, \dots$ とする. $S_n \geq 600$ となる最小の正の整数 n を求めよ. <2003年 立教大>

1, 2, 4, 5, 7, 8, ...です. 入試問題なので, 手計算で答えを出せますが, 配列を使ってやってみましょう. 配列の要素数はいくつまで準備する?

5

1. 前回のおさらい

演習5-3[1] 解答例($a[20]$ は8,135,830,267)

```
#include<stdio.h>
int main(void)
{
    int n;
    double a[21];
    a[1] = 5;
    for(n=1; n<20; n++){
        a[n+1] = 3 * a[n] + 4;
    }
    printf("a[20]は %.0lf です\n", a[20]);

    return 0;
}
```

・配列の要素を21個準備して, 問題文の添字に合わせた.
※ $a[0]$ が無駄になるが, 分かり易さを優先
・数字が大きくなりそうなので, double型配列とした

%.0lfとすると, double型の整数部のみを表示

6

1. 前回のおさらい

演習5-3[2] 解答例(a[20]は5,230,176,000)

```
#include<stdio.h>
int main(void)
{
    int i, n;
    double a[21], ans=0, n_jou;
    for(n=1; n<=20; n++){
        n_jou = 1;
        for(i=1; i<=n; i++) {n_jou = n_jou * 3;}
        a[n] = n_jou - 30;
    }
    for(n=1; n<=20; n++){
        ans = ans + a[n];
    }
    printf("a[20]までの和は %.01f です\n", ans);
    return 0;
}
```

配列を使わない方法もあります

3の累乗計算

7

1. 前回のおさらい

演習5-3[3] 解答例(最小の正の整数は29)

```
#include<stdio.h>
int main(void)
{
    int k=1, n, a[41], ans=0;
    for(n=1; n<=40; n++){
        while(k%3==0) {k++;}
        a[n] = k;
        k++;
    }
    n=0;
    while(ans<600){
        n++;
        ans = ans + a[n];
    }
    printf("Sn>=600となる最小の正数は %d です\n", n);
    return 0;
}
```

a[40]まで準備した.

3で割り切れるkをスキップし、
それ以外をa[n]に格納する.

600以上になるまで繰り返し、
600以上でループを抜ける

8

1. 前回のおさらい

発展5-1 試験の統計処理プログラム

実行例

0番目の点数: 72
1番目の点数: 68
2番目の点数: 94
3番目の点数: 26
4番目の点数: 73
5番目の点数: 88
6番目の点数: 58
7番目の点数: 85
8番目の点数: 92
9番目の点数: 45

試験結果(平均点=70.10点)
0番: 72点(偏差値50.87)
1番: 68点(偏差値49.04)
2番: 94点(偏差値60.90)
3番: 26点(偏差値29.90)
4番: 73点(偏差値51.32)
5番: 88点(偏差値58.16)
6番: 58点(偏差値44.48)
7番: 85点(偏差値56.79)
8番: 92点(偏差値59.98)
9番: 45点(偏差値38.56)

10人の点数を入力し、
平均点と各人の偏差値を出力

9

1. 前回のおさらい

発展5-1 試験の統計処理プログラム

偏差値の計算と表示

全体の平均値

$$\text{偏差値}_i = 10.0 \times \frac{\text{Score}[i] - \overline{\text{Score}}}{\text{標準偏差}} + 50.0 \quad \text{※標準偏差} = \sqrt{\text{分散}}$$

補足: ルート(√)の計算

偏差値を出すためには
平均値と分散が必要

関数「sqrt」(Square root)を利用する

```
#include <stdio.h>
#include <math.h> /*必要ヘッダファイル*/

int main(void)
{
    double root2 = sqrt(2.0);
    printf("2の平方根は%f\n", root2);
    return 0;
}
```

10

1. 前回のおさらい

発展5-1 解答例

```
#include <stdio.h>
#include <math.h>
#define DATA_SIZE 10
```

ルートの計算(sqrt)をするために、
#include <math.h>と書く必要がある

```
int main(void)
```

```
{
```

```
    int Score[DATA_SIZE], i;
```

```
    double Average = 0.0, Variance = 0.0, SD; ← 標準偏差格納用
```

```
    for(i=0;i<DATA_SIZE;i++){
        printf("%d番目の点数:",i);
        scanf("%d",&Score[i]);
        Average = Average + (double)Score[i];
    }
    Average=Average/DATA_SIZE;
```

入力と平均値の計算

```
    for(i=0;i<DATA_SIZE;i++)
        { Variance = Variance + (Score[i]-Average)*(Score[i]-Average); }
    Variance = Variance/(DATA_SIZE-1);
    SD = sqrt(Variance);
```

分散と標準偏差の計算

```
    printf("¥n試験結果 (平均点=%.2f点)¥n",Average);
    for(i=0;i<DATA_SIZE;i++)
        { printf("%d番:%d点 (偏差値%.2f)¥n",i,Score[i],10.0*(Score[i]-Average)/SD+50.0); }
```

偏差値の計算と表示

```
    return 0;
```

```
}
```

11

2. 関数

数学での関数(函数)

$$f(x) = 7.2x + 3.0$$

変数「x」に値を代入することで
ある値 $f(x)$ が決まる

Cでの関数(function)

```
double function(double x)
{
    return 7.2*x+3.0;
}
```

```
int main(void)
{
    double fx = function(10.0);
    return 0;
}
```

ある処理をまとめたもの

プログラム中の特定の処理を関数化し、
プログラムを短く読みやすくできる

実は、printf() も関数の1つで、数十行
のプログラムからできている。
もし関数がいなければ、これまでの演習のプ
ログラムの幾つかは数百行になっていた。

12

2. 関数

関数の定義方法

```
戻り値の型 関数名(引数リスト)
{
    処理
    return 戻り値;
}
```

```
double function(double x)
{
    return 7.2*x+3.0;
}
```

引数(ひきすう)

- 関数内の処理に必要な様々なデータを受け取るための変数
- 複数の引数を定義することができる
- 引数に配列を使う方法は第8回

戻り値とreturn文

- 関数での結果を1つだけ返すことができる
- この値を戻り値と呼ぶ
- 戻り値は「return」文によって設定する
- 「return」文の実行直後に関数を抜ける

【その他の注意点】

- 戻り値の型は必ず正しく指定する
- 関数の中で別の関数を定義することはできない。呼び出すことはできる。

13

2. 関数

stdio.h, stdlib.h, math.hなどのヘッダファイルは、そこで多くの関数がプロトタイプ宣言されている

関数の定義位置

関数は使用前に宣言する必要がある
下記のプログラムはエラーとなる
(WARNINGになる場合もある)



プロトタイプ宣言

関数の形式(戻り値と引数)だけを先に
コンパイラに教えておく
関数の定義位置は自由に変えられる

```
int main(void)
{
    double fx = function(10.0);
    return 0;
}

double function(double x)
{
    return 7.2*x+3.0;
}
```

エラー: 未定義な関数です!
implicit declaration of function

```
double function(double x);

int main(void)
{
    double fx = function(10.0);
    return 0;
}

double function(double x)
{
    return 7.2*x+3.0;
}
```

14

2. 関数

例1: $x-y$ を求める関数

```
#include <stdio.h>
```

```
double Sub(double x, double y)
{
    double ans;
    ans = x - y;
    return ans;
}
```

引数は「,」で区切って複数宣言できる

関数内でも別の変数を宣言できる

```
int main(void)
```

```
{
    double x=5.0, y=1.0;
    printf("%f-%f = %f\n", x, y, Sub(x, y) );
    return 0;
}
```

関数の呼び出し

(戻り値がその場所に置かれるとイメージすると分かりやすい)

15

2. 関数

例2: 値を返さない/引数をとらない関数

```
#include <stdio.h>
```

```
void check(void)
{
    printf("-----ここまで動いた-----\n");
    return; /*これは省略されることも*/
}
```

```
int main(void)
```

```
{
    int a;
    a=a+1;
    check();
    return 0;
}
```

srand()関数(第12回)もこれにあたる. 引数は数値

void型

- 「空である」ということを表す特殊な型
- void型の変数は宣言できない
「void v; /*これはエラー*/」
- 引数の「void」は省略されることが多い
- 戻り値が「void」の関数の「return」文は省略されることが多い

16

2. 関数

例3:if文と関数(絶対値を返す関数)

```
#include <stdio.h>

int Abs(int val)
{
    if(val < 0)
    {
        return -val;
    }
    return val;
}

int main(void)
{
    int a = -2;
    a = Abs(a);
    printf("絶対値は%d\n", a);
    return 0;
}
```

「return」文が実行された時点で、
処理は関数の呼び出し元に帰る



「if」文などと複数の「return」文を
組み合わせることで、異なる計算
結果を返すことができる

valが負の数ならこの時点で関数を抜ける
その後の処理「return val;」は実行しない

17

2. 関数

引数と戻り値

```
#include <stdio.h>

int Abs(int val)
{
    if(val < 0) return -val;
    return val;
}

int main(void)
{
    int a = -2;
    a = Abs(a);
    printf("絶対値は%d\n", a);
    return 0;
}
```

プログラム中に関数が
あった場合の処理

関数の情報	値
戻り値	????
引数「val」	????
処理の戻る場所	mainの2行目
....	
....	

18

2. 関数

引数と戻り値

```
#include <stdio.h>
```

```
int Abs(int val)
{
    if(val < 0) return -val;
    return val;
}
```

```
int main(void)
{
    int a = -2;
    a = Abs(a);
    printf("絶対値は%d\n", a);
    return 0;
}
```

引数に値がコピー
される

関数の情報	値
戻り値	????
引数「val」	-2
処理の戻る場所	mainの2行目
....	
....	

19

2. 関数

引数と戻り値

```
#include <stdio.h>
```

```
int Abs(int val)
{
    if(val < 0) return -val;
    return val;
}
```

関数の中が
処理される

```
int main(void)
{
    int a = -2;
    a = Abs(a);
    printf("絶対値は%d\n", a);
    return 0;
}
```

関数の情報	値
戻り値	????
引数「val」	-2
処理の戻る場所	mainの2行目
....	
....	

20

2. 関数

引数と戻り値

```
#include <stdio.h>
```

```
int Abs(int val)
```

```
{
```

```
    if(val < 0) return -val;
```

```
    return val;
```

```
}
```

```
int main
```

```
{
```

```
    int a = -2;
```

```
    a = Abs(a);
```

```
    printf("絶対値は%d\n", a);
```

```
    return 0;
```

```
}
```

return文が実行されると
戻り値の格納が行われ、
呼び出された場所に処理が戻る

関数の情報	値
戻り値	2
引数「val」	-2
処理の戻る場所	mainの2行目
....	
....	

21

2. 関数

引数と戻り値

```
#include <stdio.h>
```

```
int Abs(int val)
```

```
{
```

```
    if(val < 0) return -val;
```

```
    return val;
```

```
}
```

```
int main(void)
```

```
{
```

```
    int a = -2;
```

```
    a = 2;
```

```
    printf("絶対値は%d\n", a);
```

```
    return 0;
```

```
}
```

関数のあった場所に
戻り値が置かれているか
のように処理が進む

関数の情報	値
戻り値	2
引数「val」	-2
処理の戻る場所	mainの2行目
....	
....	

22

2. 関数

変数の局所性 (次回説明するポインタを利用する理由とも関係します)

変数「x」という名前が使われている場所

先程(例1)の関数を少しだけ書き換えたもの

```
double Sub(double x, double y)
{
    x = x - y;    変数「x」を使って計算
    return x;
}

int main(void)
{
    double x=5.0,y=1.0;
    printf("%f-%f = %f\n", x, y, Sub(x,y) );
    printf("x=%f\n",x)
    return 0;
}
```

関数内でxの値を変更してしまっているが、呼び出し側(main)に影響はないのだろうか？



引数や関数内で宣言した変数は
関数毎に独立に存在している

「x=5.000000」と表示され、Sub
におけるxに関する計算結果は、
mainのxには反映されない！

23

2. 関数

変数の局所性

```
double Sub(double x,double y)
{
    x = x - y;
    return x;
}

int main(void)
{
    double x=5.0,y=1.0;
    printf("%f-%f = %f\n", x, y, Sub(x,y) );
    printf("x=%f\n",x)
    return 0;
}
```

処理の流れ

変数の宣言があった

変数の情報

名前	値
....	
(未使用)	
(未使用)	
(未使用)	
(未使用)	
(未使用)	
(未使用)	

24

2. 関数

変数の局所性

```
double Sub(double x,double y)
{
    x = x - y;
    return x;
}
```

```
int main(void)
{
    double x=5.0,y=1.0;
    printf("%f-%f = %f\n", x, y, Sub(x,y) );
    printf("x=%f\n",x)
    return 0;
}
```

メモリ上に場所が
確保される
コンパイラが変数
の作られた場所を
管理する

処理の流れ

変数の宣言があった

変数の様子

名前	値
....	
mainのx	5.0
mainのy	1.0

25

2. 関数

変数の局所性

```
double Sub(double x,double y)
{
    x = x - y;
    return x;
}
```

```
int main(void)
{
    double x=5.0,y=1.0;
    printf("%f-%f = %f\n", x, y, Sub(x,y) );
    printf("x=%f\n",x)
    return 0;
}
```

処理の流れ

関数「Sub」が
呼ばれた

変数の様子

名前	値
....	
mainのx	5.0
mainのy	1.0

26

2. 関数

変数の局所性

```
double Sub(double x,double y)
{
    x = x - y;
    return x;
}
```

```
int main(void)
{
    double x=5.0,y=1.0;
    printf("%f-%f = %f\n", x, y, Sub(x,y));
    printf("x=%f\n",x)
    return 0;
}
```

関数の初期化

処理の流れ

関数「Sub」が
呼ばれた

変数の様子

名前	値
....	
mainのx	5.0
mainのy	1.0

27

2. 関数

変数の局所性

```
double Sub(double x,double y)
{
    x = x - y;
    return x;
}
```

```
int main(void)
{
    double x=5.0,y=1.0;
    printf("%f-%f = %f\n", x, y, Sub(x,y));
    printf("x=%f\n",x)
    return 0;
}
```

関数の初期化

コンパイラはSub
とmainを区別して
変数を管理する

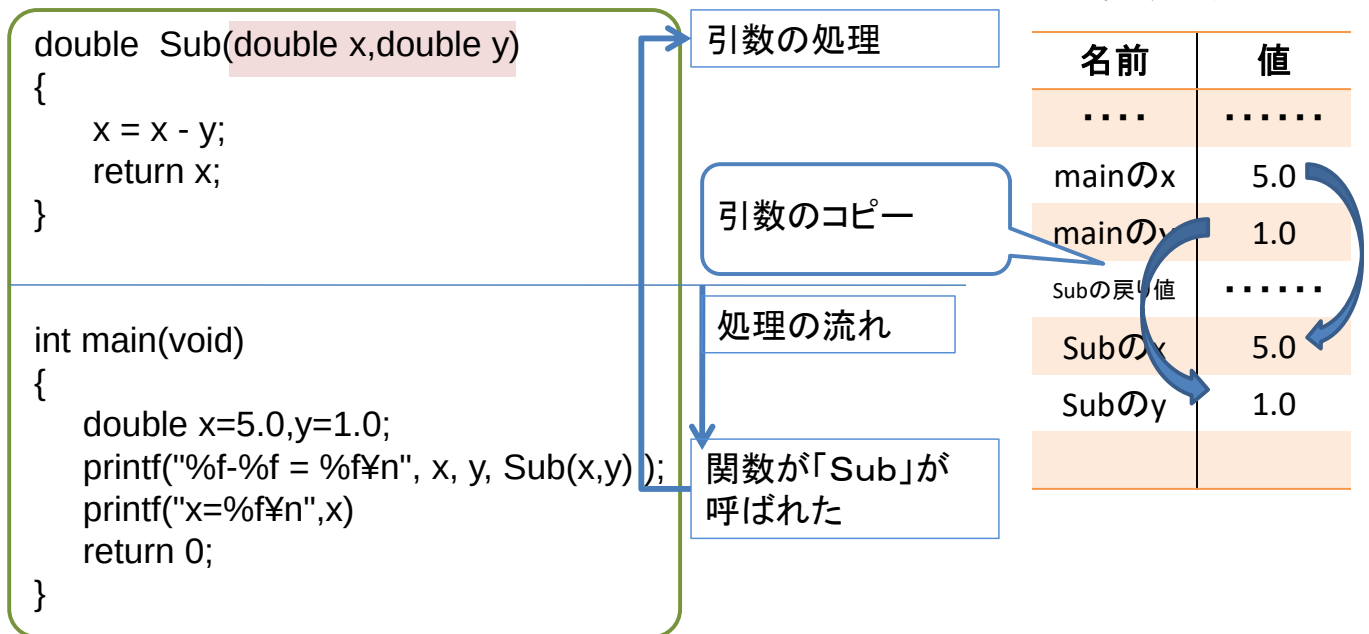
変数の様子

名前	値
....	
mainのx	5.0
mainのy	1.0
Subの戻り値	
Subのx	
Subのy	

28

2. 関数

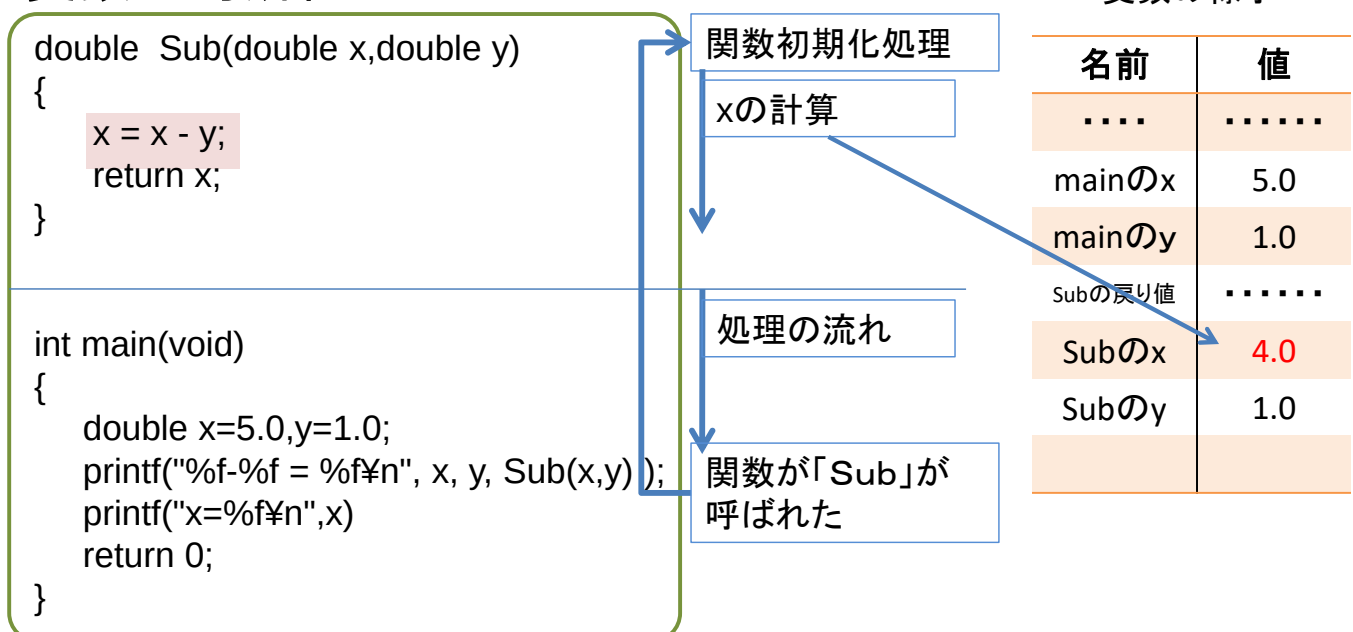
変数の局所性



29

2. 関数

変数の局所性



30

2. 関数

変数の局所性

```
double Sub(double x,double y)
{
    x = x - y;
    return x;
}
```

```
int main(void)
{
    double x=5.0,y=1.0;
    printf("%f-%f = %f\n", x, y, Sub(x,y));
    printf("x=%f\n",x)
    return 0;
}
```

関数初期化処理

xの計算

return処理①

処理の流れ

関数が「Sub」が
呼ばれた

変数の様子

名前	値
....	
mainのx	5.0
mainのy	1.0
Subの戻り値	4.0
Subのx	4.0
Subのy	1.0

31

2. 関数

変数の局所性

```
double Sub(double x,double y)
{
    x = x - y;
    return x;
}
```

```
int main(void)
{
    double x=5.0,y=1.0;
    printf("%f-%f = %f\n", x, y, Sub(x,y));
    printf("x=%f\n",x)
    return 0;
}
```

関数初期化処理

xの計算

return処理②

処理の流れ

関数が「Sub」が
呼ばれた

変数の様子

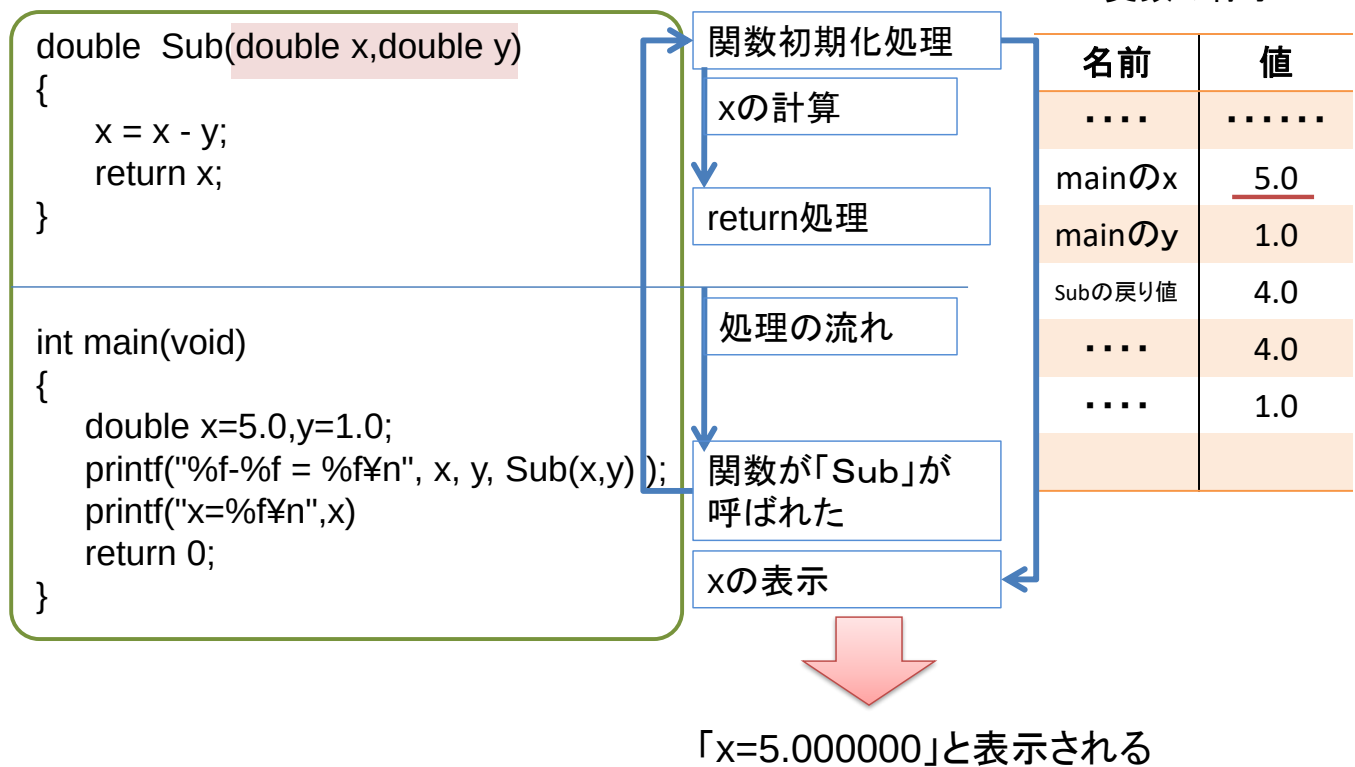
名前	値
....	
mainのx	5.0
mainのy	1.0
Subの戻り値	4.0
....	4.0
....	1.0

関数から抜ける時に、
変数は破棄される

32

2. 関数

変数の局所性



33

ここから演習課題です

34

3. 演習課題

演習6-1 関数の動作確認①

```
#include <stdio.h>

double Sub(double x, double y)
{
    double ans;
    ans = x - y;
    return ans;
}

int main(void)
{
    double x=5.0,y=1.0;
    printf("%f-%f = %f¥n", x, y, Sub(x,y) );
    return 0;
}
```

このとおりプログラムし実行してみよう。
Sub(x, y)の部分に、関数での処理結果
が入ることを確認しよう。

35

3. 演習課題

演習6-2 関数の動作確認②

```
#include <stdio.h>

void check(void)
{
    printf("-----ここまで動いた-----¥n");
}

int main(void)
{
    printf("1行目¥n");
    printf("2行目¥n");
    check();
    printf("3行目¥n");
    return 0;
}
```

main関数内にprintf文を複数作成し、左の
ように、途中にcheck()を入れて、実行させよう。
check()の位置も変えてみよう

プログラムが長くなると、「どこまで正しく動作している」
かを確認する必要があります。そのときに便利です。

36

3. 演習課題

演習6-3 関数の作成練習①

整数a, bを入力し, aのb乗を計算する関数powerを作成し, 実行してみよう.

例: a=3, b=3のとき, 関数powerの引数としてa, bを読み, aのb乗を戻り値にする

5の2i+1乗(マチンの公式): 演習課題 4-5

```
for(j=1, n_jou=1; j<=2*i+1; j++) {n_jou = n_jou * 5;}
```

3のn乗(数列): 演習課題 5-3[2]

```
n_jou = 1;  
for(i=1; i<=n; i++) {n_jou = n_jou * 3;}
```

C言語にはべき乗を計算する関数powが用意されています.

double pow(double x, double y);

math.h をインクルードすると利用できます.

自作の関数powerと, powによる計算結果を比較しましょう.

37

3. 演習課題

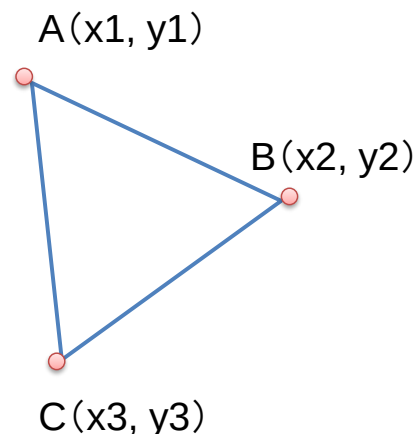
発展6-1 関数の作成練習②: 三角形の面積を求める関数

XY平面上の3点A, B, Cについて, 各点のx座標, y座標を入力し, 三角形ABCの面積を求める関数を作成してみよう.

```
double GetArea3(double x1, double y1,  
                double x2, double y2,  
                double x3, double y3)
```

```
{  
    .....  
}
```

$$\text{面積} = \frac{1}{2} |AB| |AC| \sin(\angle CAB)$$



※ヒント: 以下の値が必要

• sin∠CAB → 右参照

• 各辺の長さ → 次のスライド参照

余弦定理でcosを求め, そこからsinを計算

$$|BC|^2 = |AB|^2 + |AC|^2 - 2|AB||AC|\cos(\angle CAB)$$

38

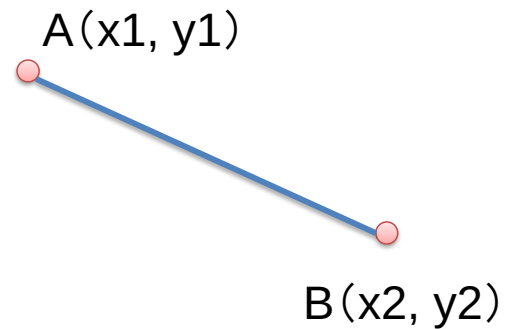
3. 演習課題

発展6-1 関数の作成練習②: 三角形の面積を求める関数

短い処理でも、よく使う処理を関数にしておくことで
プログラムを見やすくできる

2点間の距離を求める関数

```
double GetLength(double x1, double y1,  
                  double x2, double y2)  
{  
    .....  
}
```



39

今回の説明内容はおしまいです。
課題に取り組んでください。